TWEETSENSE

Project Execution Flow & Technical Interview Recall Booklet

EXECUTIVE REVISION FRAMEWORK

Principle: FLOW → WHAT HAPPENS → TECHNOLOGY USED → WHY IT IS USED → TECHNICAL RECALL → LIMITATION / TRADE-OFF

Mentally walk through the application running end-to-end from browser interaction to classification, enrichment, and UI response.

STAGE 0 | System Initialization & Server Boot

- **Action:** Flask application initializes on port 5000 (analysis_backend.py) prior to serving any user requests.
- **Memory Initialization:** Pre-trained model artifacts are deserialized into RAM once at startup via joblib.load():
  - **vectorizer.sav:** 16.5 MB TF-IDF vocabulary matrix learned during offline training.
  - **trained_model.sav:** 5.4 MB Logistic Regression weights.
- **NLP Pipeline Warmup:** spaCy en_core_web_sm loads with parser and NER disabled to minimize latency and memory. Hugging Face zero-shot pipeline warms up facebook/bart-large-mnli.
- **Why Here:** Eliminates 3–8 seconds of repetitive disk I/O per HTTP request; models reside warm in RAM for immediate inference.
- **Trade-off / Challenge:** Higher server baseline memory footprint (~1.5–2 GB) and slower cold start time.
- **Interviewer Recall:** Stateless API with in-memory model caching; prevents per-request file I/O bottlenecks.

STAGE 1 | User Input Capture (Frontend Interface)

- **Action:** User opens the web interface running on Next.js 15 (port 3000) and types or pastes tweet text into the search input.
- **State Management:** React keeps the query in local component state: searchQuery via useState<string>('').
- **Trigger:** User clicks the 'Analyze' button, firing the handleAnalysis event handler.
- **Technology Used:** Next.js 15, React 19, TypeScript, Tailwind CSS v4.
- **Why Here:** Provides immediate responsive user interaction and typed component state binding.
- **Technical Recall:** Controlled component pattern; SyntheticEvent dispatch; zero page reload (SPA architecture).

STAGE 2 | Client-Side Input Validation & State Transition

- **Action:** handleAnalysis validates input length before initiating network transmission: searchQuery.trim().length >= 5.
- **Invalid Handling:** If < 5 characters, sets inline error state immediately and aborts execution, saving backend compute.
- **Valid Handling:** Clears previous errors, sets loading = true, and resets previous result state.
- **Why Here:** Defensive frontend engineering; avoids sending garbage or single-character strings to heavy transformer/ML pipelines.

- **Technical Recall:** Client-side guard clause; asynchronous UI loading state transitions.

STAGE 3 | Network Transport & API Request Dispatch

- **Action:** Frontend dispatches an asynchronous HTTP POST request using fetch():
  - **Endpoint:** http://localhost:5000/api/analyze
  - **Headers:** Content-Type: application/json
  - **Body:** JSON.stringify({ text: searchQuery })
- **Why Here:** Strictly decouples the Next.js presentation layer from the Python ML execution environment.
- **Technical Recall:** RESTful POST contract; JSON payload serialization; cross-port communication (3000 → 5000).

STAGE 4 | Flask Endpoint Reception & CORS Handling

- **Action:** Flask receives request on route @app.route('/api/analyze', methods=['POST']) and extracts JSON via request.get_json().
- **Validation:** If 'text' key is absent, backend returns HTTP 400 Bad Request with an error description.
- **CORS Interception:** flask-cors handles cross-origin HTTP headers allowing requests from port 3000.
- **Challenge & Solution:** Browser Same-Origin Policy (SOP) blocks requests across different ports. Solved via CORS(app) appending Access-Control-Allow-Origin: *.
- **Limitation:** CORS(app) is globally open for development; production must restrict origin to the specific frontend domain.
- **Technical Recall:** HTTP Request lifecycle; Same-Origin Policy tuple (protocol, host, port); HTTP status codes (200, 400, 500).

STAGE 5 | Linguistic Text Preprocessing (spaCy Lemmatization)

- **Action:** Raw text string is passed into preprocess_text(user_input).
- **NLP Pipeline:** spaCy en_core_web_sm tokenizes the string and extracts the lemma of each token: ' '.join([token.lemma_ for token in doc]).
- **Why Here:** Reduces inflected and conjugated forms of words to their dictionary root (e.g., 'running', 'ran' → 'run'), consolidating feature space.
- **Evolution:** Exploration began with Porter Stemming in proj.ipynb, but switched to spaCy lemmatization for accurate grammatical normalization.
- **Critical Parity Rule:** Inference preprocessing must be identical to training preprocessing; mismatched token transformations break vocabulary mapping.
- **Technical Recall:** Stemming (heuristic suffix stripping) vs. Lemmatization (morphological dictionary lookup); spaCy efficiency optimizations.
- **Interviewer Check:** Why lemmatization over stemming? Why must inference preprocessing match training preprocessing identically?

STAGE 6 | Numerical Feature Extraction (TF-IDF Vectorization)

- **Action:** Lemmatized string is transformed into a high-dimensional numerical feature vector: X = vectorizer.transform([processed_text]).
- **Crucial Distinction:** Uses transform(), NEVER fit_transform(). fit_transform() would wipe out the trained 1.6M-tweet vocabulary.

- **Technology Used:** scikit-learn TfidfVectorizer loaded from vectorizer.sav.
- **Why Here:** Logistic Regression cannot process natural text strings; TF-IDF converts text into sparse numeric feature coordinates.
- **How TF-IDF Works:** Term Frequency (frequency in current tweet) × Inverse Document Frequency (penalizes common words across the corpus, log(N/DF)).
- **Trade-off:** Bag-of-words representation completely ignores word order, grammatical syntax, and negation positioning.
- **Technical Recall:** scipy.sparse.csr_matrix representation; vocabulary index lookup; out-of-vocabulary (OOV) tokens ignored.

STAGE 7 | Core Sentiment Classification & Confidence Scoring

- **Action:** Feature matrix X is fed to the classifier: prediction = model.predict(X)[0] (0 = Negative, 1 = Positive).
- **Confidence Calculation:** proba = model.predict_proba(X)[0], confidence = max(proba) (e.g., 0.9321 certainty).
- **Technology Used:** scikit-learn LogisticRegression(max_iter=1000) loaded from trained_model.sav.
- **Why Logistic Regression:** Extremely fast inference on CPU (simple dot product + sigmoid), highly interpretable, robust with high-dimensional sparse TF-IDF vectors.
- **Why Not BERT:** BERT requires GPU inference for low latency, takes massive RAM, and adds orchestration overhead; LR delivers ~79.8% test accuracy in milliseconds.

> **ML BRIDGE: OFFLINE TRAINING VS. ONLINE INFERENCE**
>
> *OFFLINE TRAINING PHASE (proj.ipynb):*
> • *Dataset: Sentiment140 (Stanford benchmark of 1,600,000 tweets; 0 = Negative, 4 = Positive mapped to 1).*
> • *Preprocessing: 1.6M tweets processed through spaCy lemmatization (saved as twitter_data_lemmatized.csv).*
> • *Vectorizer: TF-IDF fitted across all 1.6M records to establish the permanent global vocabulary.*
> • *Model Training: Logistic Regression fit on 80% train split (max_iter=1000).*
> • *Verified Project Metrics:*
> - *Training Accuracy: ~81.7%*
> - *Testing Accuracy: ~79.8%*
> - *Weighted F1-Score: ~0.804*
> • *Artifact Export: joblib.dump() outputs trained_model.sav (5.4 MB) and vectorizer.sav (16.5 MB).*
>
> *ONLINE INFERENCE PHASE (analysis_backend.py):*
> • *Zero model fitting occurs. The saved model performs static inference scoring in single-digit milliseconds.*

STAGE 8 | Three-Class Sentiment Heuristic (TextBlob Polarity Check)

- **Action:** get_sentiment_label(user_input, prediction) checks TextBlob polarity: polarity = TextBlob(text).sentiment.polarity.
- **Decision Rule:** If -0.1 <= polarity <= 0.1 → Overrides output to 'Neutral'. Otherwise, assigns 'Negative' (if 0) or 'Positive' (if 1).
- **Engineering Challenge:** Sentiment140 is strictly binary (Positive/Negative). Real tweets contain factual announcements and neutral statements.
- **Solution:** Constructed a hybrid classifier: Statistical ML handles binary classification, while TextBlob lexicon polarity filters out neutral text.

- **Limitation / Trade-off:** Neutral is a post-processing heuristic rule, not a statistically learned class trained on ground-truth neutral data.
- **Technical Recall:** Rule-based lexicon scoring (-1.0 to +1.0) vs. statistical supervised learning; hybrid decision boundary.
- **Interviewer Check:** Is Neutral trained by the Logistic Regression model? (Answer: No, it is a post-processing heuristic via TextBlob).

STAGE 9 | Priority & Urgency Tagging (BART Zero-Shot Classification)

- **Action:** prioritize_comment(user_input) passes text to Hugging Face pipeline('zero-shot-classification').
- **Model Used:** facebook/bart-large-mnli with candidate labels: ['High Priority', 'Medium Priority', 'Low Priority'].
- **Label Selection:** Model scores probability of entailment across candidates; top label selected via result['labels'][0].
- **Why Here:** Flags urgent customer support issues, product crashes, or high-escalation complaints automatically.
- **Why Zero-Shot:** No labeled priority dataset existed. Natural Language Inference (NLI) formulations allow pretrained models to score arbitrary labels without retraining.
- **Trade-off / Bottleneck:** BART-large is the slowest operation in the backend pipeline (~0.5s–2.0s latency on CPU).
- **Technical Recall:** Zero-shot premise-hypothesis entailment scoring; fallback returns 'N/A' if transformer fails.

STAGE 10 | Extractive Text Summarization (Sumy LSA)

- **Action:** get_summary(user_input, num_sentences=2) generates a concise text summary.
- **Technology Used:** Sumy library with LsaSummarizer (Latent Semantic Analysis) and PlaintextParser.
- **Why Here:** Provides users an instant executive summary of multi-sentence tweets or extended feedback text.
- **Design Evolution:** Legacy app2.py used heavy transformer-based Pegasus summarization; analysis_backend.py switched to Sumy LSA for lightweight mathematical extractive summarization.
- **Technical Recall:** Extractive summarization (extracting key existing sentences via SVD) vs. Abstractive summarization (generating new text).

STAGE 11 | In-Memory Word Cloud Generation & Base64 Encoding

- **Action:** generate_wordcloud(user_input) generates a word frequency visual via WordCloud(width=800, height=400, max_words=100).
- **In-Memory Storage:** Image is written directly to an in-memory byte buffer via io.BytesIO() rather than writing a file to disk.
- **Base64 Encoding:** Buffer is Base64-encoded to produce a Data URI string: f'{img_str}'.
- **Challenge & Solution:** REST APIs cannot easily transmit binary PNG files inside JSON without multi-part protocols. Solved by encoding the image into Base64 ASCII text inside the JSON payload.
- **Benefit:** Completely eliminates disk I/O bottlenecks, storage leak cleanups, and local file permission issues.
- **Technical Recall:** Binary-to-ASCII encoding (33% size overhead); in-memory streams; Data URI browser rendering.

STAGE 12 | Response Packaging & HTTP Serialization

- **Action:** Flask consolidates all computed results into a single dictionary and executes jsonify(response):

    - **status:** 'success'
    - **sentiment:** 'Positive' | 'Negative' | 'Neutral'
    - **confidence:** Formatted string (e.g., '0.9412')
    - **priority:** 'High Priority' | 'Medium Priority' | 'Low Priority'
    - **input_length:** Word count integer
    - **summary:** Extractive summary string
    - **wordcloud_img:** Base64 data URL string
- **Status Codes:** Returns HTTP 200 OK on success; uncaught exceptions return HTTP 500 Internal Server Error.
- **Architecture:** Completely stateless API; zero database transactions; zero session persistence.

STAGE 13 | Client State Reconciliation & UI Rendering

- **Action:** Next.js receives HTTP response: const data = await response.json().
- **State Updates:** Executes setResult(data as AnalysisResult) and setLoading(false).
- **DOM Reconciliation:** React re-renders the component tree with full animation:
    - **Sentiment Badge:** Color coded (Positive → Emerald Green, Negative → Crimson Red, Neutral → Blue).
    - **Confidence Donut:** Rendered via Recharts Pie component showing model certainty.
    - **Priority Badge:** Highlights ticket urgency level.
    - **Word Cloud:** Rendered natively via directly from the Base64 string.
- **UX Polish:** Smooth scroll auto-navigates to #results.
- **Technical Recall:** React virtual DOM reconciliation; Recharts SVG rendering; zero-latency Base64 image decoding.

STAGE 14 | Batch CSV Processing Branch (/api/batch_analyze)

- **Trigger:** User uploads a CSV file containing a mandatory 'text' column.
- **Transport:** Next.js packages file into FormData and POSTs to /api/batch_analyze.
- **Backend Iteration:** Flask reads file into memory via pandas (pd.read_csv(file)). Iterates through each record:
    - **Per-Row Inference:** Executes lemmatization → TF-IDF transform → LR predict → TextBlob → BART priority.
- **Dataset Aggregation:** pandas calculates comprehensive dataset metrics:
    - **Distributions:** Sentiment counts and Priority counts dictionaries.
    - **Averages:** Average confidence score and average word count.
    - **Global Insights:** 3-sentence combined summary + global combined Word Cloud Base64 string.
- **Client Presentation:** Frontend renders Recharts distribution charts, paginated tabular data, and enables direct client-side CSV download.
- **Limitation:** Sequential Python loop; large CSV files (>5,000 rows) will cause HTTP gateway timeout. Improvement: Asynchronous worker queue (Celery + Redis).

Walk Me Through TweetSense (Interview Speaking Script)

> **VERBATIM INTERVIEW WALKTHROUGH SCRIPT**
> *"TweetSense is an end-to-end sentiment intelligence platform that classifies tweet text into Positive, Negative, or Neutral sentiment while generating urgency priority tags, extractive summaries, and word cloud visualizations. The*

> *architecture consists of a Next.js 15 client communicating with a stateless Flask REST API. When a user submits text, the frontend validates the input and dispatches a JSON POST request. Flask receives the text and passes it to spaCy for lemmatization, stripping inflectional endings while matching the exact preprocessing used during model training. The normalized string is transformed into a sparse feature vector using a pre-fitted TF-IDF vectorizer trained on 1.6 million Sentiment140 tweets. A Logistic Regression model predicts the binary sentiment and outputs a confidence score. To handle neutral tweets without labeled 3-class training data, we apply a hybrid heuristic: if TextBlob polarity falls between -0.1 and 0.1, the result is assigned as Neutral. Concurrently, a zero-shot BART transformer classifies the comment's priority level, Sumy LSA extracts key summary sentences, and WordCloud generates an in-memory PNG encoded as a Base64 data URL. The consolidated results return in a single JSON payload to Next.js, which updates component state and renders sentiment badges, Recharts visualizations, and the word cloud without any database dependencies."*

TweetSense in 30 Seconds (Quick-Reference Spec Table)

Problem	Manual review of high-volume social media streams is slow and subjective.
Input	Raw text query or uploaded CSV file with a mandatory 'text' column.
Frontend	Next.js 15, React 19, TypeScript, Tailwind CSS v4, Recharts (Port 3000).
API & Backend	Flask REST API, CORS enabled, stateless in-memory execution (Port 5000).
Dataset	Sentiment140 (Stanford benchmark of 1,600,000 tweets; binary 0 and 4).
Preprocessing	spaCy en_core_web_sm lemmatization (parser & NER disabled).
Main Classifier	scikit-learn LogisticRegression(max_iter=1000) serialized via joblib.
Feature Vectorizer	scikit-learn TfidfVectorizer (sparse vocabulary matrix).
Neutral Handling	TextBlob polarity heuristic ([-0.1, 0.1]) overriding binary LR prediction.
Priority Tagging	Hugging Face facebook/bart-large-mnli zero-shot classification.
Summarization	Sumy LSA (Latent Semantic Analysis) extractive summarizer (2–3 sentences).
Visualization	In-memory WordCloud via io.BytesIO() encoded to Base64 Data URI.
Data Persistence	None — completely stateless; batch results exported client-side as CSV.
Verified Metrics	~79.8% Test Accuracy | ~81.7% Train Accuracy | ~0.804

	Weighted F1.
Biggest Limitation	BART zero-shot introduces CPU latency; Neutral class is an unlearned heuristic.
Key Improvement	Fine-tune a compact BERT model on native 3-class data to unify sentiment and priority.

12 Must-Remember High-Yield Facts

1. **Dataset:** Sentiment140 from Stanford (1.6M tweets; binary: 0 = Negative, 4 = Positive mapped to 1).
2. **Classifier:** Logistic Regression trained with scikit-learn; serialized via joblib into trained_model.sav (5.4 MB).
3. **Features:** TF-IDF vectorizer serialized into vectorizer.sav (16.5 MB); vocabulary learned once during offline training.
4. **Verified Metrics:** ~79.8% testing accuracy, ~81.7% training accuracy, and 0.804 weighted F1-score.
5. **Preprocessing:** spaCy lemmatization (en_core_web_sm), NOT Porter stemming (stemming was explored first, then replaced).
6. **Preprocessing Parity:** Preprocessing at inference must be identical to training preprocessing to avoid feature mismatch.
7. **Neutral Trick:** Sentiment140 has no neutral label; TextBlob polarity [-0.1, 0.1] overrides model output to Neutral.
8. **Zero-Shot Priority:** Uses facebook/bart-large-mnli via Hugging Face pipeline for High, Medium, Low Priority labels.
9. **Extractive Summary:** Generated using Sumy LSA summarizer; avoids heavy abstractive sequence generation models.
10. **Zero Disk I/O for Images:** WordCloud renders into BytesIO RAM buffer and returns as a Base64 data string in JSON.
11. **Networking Ports:** Next.js runs on port 3000; Flask runs on port 5000; flask-cors bridges the cross-origin boundary.
12. **Persistence Architecture:** No SQL/NoSQL database; application is stateless; batch results download directly to client CSV.